

Chapter 12

Graph-Matrix Calculus for Computational Convex Analysis

Bryan Gardiner and Yves Lucet

Abstract We introduce a new family of algorithms for computing fundamental operators arising from convex analysis. The new algorithms rely on the fact that the graph of the subdifferential of most convex operators depends linearly on the graph of the subdifferential of the function. By storing the subdifferential information, the computation of the conjugate is reduced to a matrix multiplication. We explain how other operators can be computed similarly, and present numerical experiments that compare graph-matrix calculus algorithms with piecewise-linear quadratic algorithms from computational convex analysis (CCA), and with a bundle method using warmstarting. Our results show that the new algorithms are an order of magnitude faster. They also add subdifferential calculus to our numerical library, and are very simple to implement.

Keywords Computer-Aided convex analysis · Computational convex analysis · Convex function · Fenchel conjugate · Legendre–Fenchel transform · Proximal average · Proximal mapping · Subdifferential operator.

AMS 2010 Subject Classification: 90C25, 26A51, 26B25, 47H05, 52A41

12.1 Introduction

The birth of computational convex analysis (CCA) algorithms can be traced back to the introduction of fast algorithms to compute the (Legendre–Fenchel) conjugate [7, 9, 17, 25, 27], although key ideas were introduced much earlier [24, Paragraph 5c]. While other transforms such as the Moreau–Yosida approximate can be deduced from these algorithms, the study of monotone operators

Y. Lucet (✉)

Computer Science, I. K. Barber School, University of British Columbia Okanagan,
Kelowna, B.C. V1V 1V7, Canada
e-mail: yves.lucet@ubc.ca

using the Rockafellar and Fitzpatrick functions require specialized algorithms [11]. The CCA numerical library provides a numerical implementation of most of these algorithms and is available from [8].

The performance of algorithms in CCA [17–21] was challenged with the introduction of the proximal average. The origin of this operator can be traced back to Moreau [24] when he proved that the set of proximal mappings is convex. The proximal average was explicitly defined in 2004 [2]. It has been studied in [3–6, 16, 23], and extended in [1, 13]. Its numerical computation is challenging as it requires computing the composition of several operators. It is currently performed with a family of algorithms based on the class of piecewise linear-quadratic (PLQ) functions [22].

The linear relationship between the graph of the subdifferential of the most common convex analysis operators such as the (Legendre–Fenchel) conjugate was noticed in [12] while basing computational algorithms on such properties was suggested in [21]. By storing the subdifferential data, the computation of the conjugate is reduced to a multiplication by a 2×2 matrix. We show that other operators can be computed similarly, and compare numerical algorithms based on graph-matrix calculus (named GPH algorithms in the following, GPH standing for graph) with PLQ algorithms (which we call PLQ algorithms) introduced in [22], and with the linear-time Legendre transform (LLT) algorithm (named LLT algorithm in the following) from [18]. We also compare them with warmstarting using a bundle method (named OPT algorithms in the paper). Graph-matrix calculus algorithms are particularly well suited for implementation in mathematical software like Matlab and Scilab, which are optimized for matrix operations but penalize heavily the usage of loops. They naturally extend known numerical algorithms by providing easy manipulation of the subdifferential.

The paper is organized as follows. Section 12.1 introduces the paper; Section 12.2 fixes the notations and recalls the definitions of the main operators in convex analysis including two self-dual smoothing operators. Section 12.3 recalls Goebel’s graph-matrix calculus [12, p. 181] and provides the matrices associated with most unary and binary operators. Section 12.4 proposes a GPH data structure, which is used in Sect. 12.5 to present linear-time algorithms for the main convex operators. In Sect. 12.6, we compare the running time of PLQ, GPH, OPT, and LLT algorithms and emphasize the advantages of the GPH algorithms. Finally, Sect. 12.7 summarizes our results and proposes future directions.

12.2 Preliminaries

First, we fix our notations. We will adopt Matlab and Scilab matrix notation to write the matrix

$$G = \begin{bmatrix} x \\ s \\ y \end{bmatrix}, \quad (12.1)$$

where x , s , and y are row vectors as $G = [x; s; y]$.

We consider functions $f : \mathbb{R}^d \rightarrow \mathbb{R} \cup \{+\infty\}$. The set of points where the function takes finite values is called the effective domain and denoted $\text{dom } f$. A function f is proper if $\text{dom } f$ is nonempty. Unless otherwise stated all functions considered are proper lower semicontinuous (lsc) convex. We denote by $\mathbb{R}^+$ the set of positive real numbers, $\text{ri } S$ the relative interior of a set S , $\text{Id} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ the identity mapping, $\|\cdot\|$ the Euclidean norm, $\langle \cdot, \cdot \rangle$ the standard dot product, and $\square$ the inf-convolution operator defined by

$$f \square g(x) = \inf_{y \in \mathbb{R}^d} [f(y) + g(x - y)],$$

while

$$\partial f(x) = \{s \in \mathbb{R}^d : \forall y \in \mathbb{R}^d, f(y) \geq f(x) + \langle s, y - x \rangle\}$$

denotes the convex subdifferential, and

$$\text{gph } \partial f = \{(x, s) \in \mathbb{R}^d \times \mathbb{R}^d : s \in \partial f(x)\}$$

the graph of the subdifferential. More generally, we denote $\text{gph } P$ the graph of an operator $P : \mathbb{R}^d \rightrightarrows \mathbb{R}^d$, i.e.,

$$\text{gph } P = \{(x, s) \in \mathbb{R}^d \times \mathbb{R}^d : s \in P(x)\}.$$

We write the (Legendre–Fenchel) conjugate as

$$f^*(s) = \sup_{x \in \mathbb{R}^d} [\langle s, x \rangle - f(x)],$$

and the Moreau(-Yosida) envelope of a function f with parameter $\lambda > 0$ as

$$e_\lambda f(x) = (f \square \lambda^{-1} q)(x) = \min_{y \in \mathbb{R}^d} \left[f(y) + \frac{\|x - y\|^2}{2\lambda} \right],$$

where q is the quadratic kernel $q(x) = \|x\|^2/2$. The proximal mapping is the application that assigns to a point x the minimizers in the definition of the Moreau envelope

$$\text{Prox}_\lambda f(x) = \underset{y \in \mathbb{R}^d}{\text{Argmin}} \left[f(y) + \frac{\|x - y\|^2}{2\lambda} \right].$$

The epi-multiplication operator is defined by

$$\alpha \star f = \begin{cases} \alpha f(\cdot/\alpha) & \text{if } \alpha > 0, \\ \iota_{\{0\}} & \text{if } \alpha = 0, \end{cases}$$

where $\iota_{\{0\}}$ is the indicator function: $\iota_{\{0\}}(x) = 0$ if $x = 0$ otherwise it is $+\infty$.

The *proximal average* is the function $\mathcal{P}(f_0, f_1; \lambda_0, \lambda_1) : \mathbb{R}^d \rightarrow]-\infty, +\infty]$ defined by

$$\mathcal{P}(f_0, f_1; \lambda_0, \lambda_1) = \left(\lambda_0 \left(f_0 + \frac{1}{\mu} q \right)^* + \lambda_1 \left(f_1 + \frac{1}{\mu} q \right)^* \right)^* - \frac{1}{\mu} q,$$

where the functions $f_0, f_1 : \mathbb{R}^d \rightarrow]-\infty, +\infty]$ are proper lsc convex functions, $\mu > 0$, and $\lambda_0, \lambda_1 > 0$ with $\lambda_0 + \lambda_1 = 1$. When there is no ambiguity on $f_0, f_1, \lambda_0, \lambda_1$, we will simplify the notation to $\mathcal{P}$. Note that the proximal average can be written as the constrained minimization problem [5, Formula (20)]

$$\mathcal{P}(f_0, f_1; \lambda_0, \lambda_1)(\xi) = \inf_{\lambda_0 y_0 + \lambda_1 y_1 = \xi} \left[\lambda_0 f_0(y_0) + \lambda_1 f_1(y_1) + \frac{\lambda_0 \lambda_1}{2\mu} \|y_0 - y_1\|^2 \right],$$

which is the basis to generalizing the proximal average as the kernel average [1]. When the infimum in the definition of the proximal average is reached at y_0, y_1 , we say the proximal average is *exact* at $\xi = \lambda_0 y_0 + \lambda_1 y_1$. We will use the following formula later [5, Proposition 4.3]

$$\mathcal{P}(f_0, f_1; \lambda_0, \lambda_1)(\xi) = \inf_{\lambda_0 y_0 + \lambda_1 y_1 = \xi} \left[\sum_{i=0}^1 (\lambda_i (f_i(y_i) + q(y_i))) - q(\xi) \right]. \quad (12.2)$$

The proximal average is a very useful convex operator as it is a convex function [5, Corollary 5.2] that satisfies $\mathcal{P}(f_0, f_1; \lambda_0, \lambda_1) = -e_\mu(-\lambda_0 e_\mu f_0 - \lambda_1 e_\mu f_1)$ [13, Formula (4)], [5, Theorem 8.3], and $\text{dom } \mathcal{P}(f_0, f_1; \lambda_0, \lambda_1) = \lambda_0 \text{dom } f_0 + \lambda_1 \text{dom } f_1$ [5, Theorem 4.6]. Moreover, the proximal mapping of the proximal average is the convex combination of the proximal mappings: $\text{Prox } \mathcal{P}(f_0, f_1; \lambda_0, \lambda_1) = \lambda_0 \text{Prox } f_0 + \lambda_1 \text{Prox } f_1$ [5, Theorem 6.7], and the conjugate of the proximal average is the proximal average of the conjugate [5, Theorem 5.1]

$$[\mathcal{P}(f_0, f_1; \lambda_0, \lambda_1)]^* = \mathcal{P}(f_0^*, f_1^*; \lambda_0, \lambda_1).$$

Finally, we recall the self-dual smoothing operator

$$s_\lambda f = (1 - \lambda^2) e_\lambda f + \lambda q,$$

introduced in [12], and another self-dual operator

$$T_\lambda f = \mathcal{P}(f, q; 1 - \lambda, \lambda)$$

introduced in [6, 23].

We will work with PLQ functions, which were described in [26] and implemented in [22]. For univariate functions, they are extended-valued piecewise quadratic functions defined on a finite number of intervals which are continuous on the interior of their effective domain. In other words, they are defined for $x \in [x_i, x_{i+1}]$ by $p(x) = a_i x^2 + b_i x + c_i$, where $a_i, b_i \in \mathbb{R}$, $c_i \in \mathbb{R} \cup \{+\infty\}$, $x_i \in \mathbb{R} \cup \{-\infty, +\infty\}$ with $-\infty = x_0 < x_1 < \dots < x_n < x_{n+1} = +\infty$.

12.3 Goebel's Graph-Matrix Calculus

We now collect rules, most of them introduced in [12, p. 181], on how the graph of the subdifferential mapping ∂f is transformed under basic operations on f .

Assume $f : \mathbb{R}^d \rightarrow \mathbb{R} \cup \{+\infty\}$ is a proper lsc convex function. The following calculus rules, which we call Goebel's Graph-matrix calculus for unary operators, on the graph of the subdifferential hold for any $\alpha > 0$, $0 < \lambda < 1$, and $\beta \geq 0$.

$$\begin{aligned}
 \text{gph } \partial(f^*) &= \begin{bmatrix} 0 & \text{Id} \\ \text{Id} & 0 \end{bmatrix} \text{gph } \partial f & \text{gph } \partial(\alpha f) &= \begin{bmatrix} \text{Id} & 0 \\ 0 & \alpha \text{Id} \end{bmatrix} \text{gph } \partial f \\
 \text{gph } \partial(\alpha \star f) &= \begin{bmatrix} \alpha \text{Id} & 0 \\ 0 & \text{Id} \end{bmatrix} \text{gph } \partial f & \text{gph } \partial(f(\alpha \cdot)) &= \begin{bmatrix} \alpha^{-1} \text{Id} & 0 \\ 0 & \alpha \text{Id} \end{bmatrix} \text{gph } \partial f \\
 \text{gph } \partial(f + \beta q) &= \begin{bmatrix} \text{Id} & 0 \\ \beta \text{Id} & \text{Id} \end{bmatrix} \text{gph } \partial f & \text{gph } \partial e_\alpha(f) &= \begin{bmatrix} \text{Id} & \alpha \text{Id} \\ 0 & \text{Id} \end{bmatrix} \text{gph } \partial f \\
 \text{gph } \text{Prox}_\alpha(f) &= \begin{bmatrix} \text{Id} & \alpha \text{Id} \\ \text{Id} & 0 \end{bmatrix} \text{gph } \partial f & \text{gph } \partial s_\lambda f &= \begin{bmatrix} \text{Id} & \lambda \text{Id} \\ \lambda \text{Id} & \text{Id} \end{bmatrix} \text{gph } \partial f.
 \end{aligned} \tag{12.3}$$

All these formulas follow directly from well-known convex analysis formulas except the last one, which was proven in [12, p. 181] based on the definition of $s_\lambda f$. The operator T_λ admits the following formula.

Lemma 12.1. *Under the above assumptions we have*

$$\text{gph } \partial T_\lambda f = \begin{bmatrix} (1 - \frac{\lambda}{2}) \text{Id} & \frac{\lambda}{2} \text{Id} \\ \frac{\lambda}{2} \text{Id} & (1 - \frac{\lambda}{2}) \text{Id} \end{bmatrix} \text{gph } \partial f. \tag{12.4}$$

Proof. We use [23, Theorem 4.3.2(ii)]

$$T_\lambda f = (1 - \lambda) \left[f \square \left(\frac{2 - \lambda}{\lambda} \right) q \right] \left(\frac{2}{2 - \lambda} \cdot \right) + \frac{\lambda}{2 - \lambda} q$$

to deduce

$$\begin{aligned}
 \text{gph } \partial T_\lambda f &= \begin{bmatrix} \text{Id} & 0 \\ \frac{\lambda}{2-\lambda} \text{Id} & \text{Id} \end{bmatrix} \text{gph } \partial \left[(1-\lambda) \left(f \square \left(\frac{2-\lambda}{\lambda} \right) q \right) \left(\frac{2}{2-\lambda} \cdot \right) \right] \\
 &= \begin{bmatrix} \text{Id} & 0 \\ \frac{\lambda}{2-\lambda} \text{Id} & \text{Id} \end{bmatrix} \begin{bmatrix} \frac{2-\lambda}{2} \text{Id} & 0 \\ 0 & \frac{2}{2-\lambda} \text{Id} \end{bmatrix} \text{gph } \partial \left[(1-\lambda) \left(f \square \left(\frac{2-\lambda}{\lambda} \right) q \right) \right] \\
 &= \begin{bmatrix} \text{Id} & 0 \\ \frac{\lambda}{2-\lambda} \text{Id} & \text{Id} \end{bmatrix} \begin{bmatrix} \frac{2-\lambda}{2} \text{Id} & 0 \\ 0 & \frac{2}{2-\lambda} \text{Id} \end{bmatrix} \begin{bmatrix} \text{Id} & 0 \\ 0 & (1-\lambda) \text{Id} \end{bmatrix} \text{gph } \partial \left[f \square \left(\frac{2-\lambda}{\lambda} \right) q \right] \\
 &= \begin{bmatrix} \text{Id} & 0 \\ \frac{\lambda}{2-\lambda} \text{Id} & \text{Id} \end{bmatrix} \begin{bmatrix} \frac{2-\lambda}{2} \text{Id} & 0 \\ 0 & \frac{2}{2-\lambda} \text{Id} \end{bmatrix} \begin{bmatrix} \text{Id} & 0 \\ 0 & (1-\lambda) \text{Id} \end{bmatrix} \begin{bmatrix} \text{Id} & \frac{\lambda}{2-\lambda} \text{Id} \\ 0 & \text{Id} \end{bmatrix} \text{gph } \partial f.
 \end{aligned}$$

Multiplying the matrices gives the result. ■

Similar formulas hold for binary operators.

Lemma 12.2 (Graph-matrix calculus for binary operators). *Assume f_1 (resp. f_2) is a proper lsc convex function with $s_1 \in \partial f_1(x_1)$ (resp. $s_2 \in \partial f_2(x_2)$). The index i is in $\{1, 2\}$. Consider points $x \in \mathbb{R}^d$ (primal space) and $s \in \mathbb{R}^d$ (dual space).*

(i) *If $\text{ri dom } f_1 \cap \text{ri dom } f_2 \neq \emptyset$, then*

$$(x, s) \in \text{gph } \partial(f_1 + f_2) \Leftrightarrow \exists (x_i, s_i) \in \text{gph } \partial f_i \text{ such that } \begin{cases} x = x_1 = x_2, \\ s = s_1 + s_2. \end{cases}$$

(ii) *If $\partial f_1(x_1) \cap \partial f_2(x_2) \neq \emptyset$, then*

$$(x_1 + x_2, s) \in \text{gph } \partial(f_1 \square f_2) \Leftrightarrow (x_i, s) \in \text{gph } \partial f_i.$$

(iii) *Take $\lambda_1 + \lambda_2 = 1$ with $\lambda_1, \lambda_2 > 0$. Assume $\mathcal{P}_\mu(f_1, f_2; \lambda_1, \lambda_2)$ is exact (i.e. the infimum is attained) at $x = \lambda_1 x_1 + \lambda_2 x_2$ with $x_i \in \text{dom } f_i$. Then*

$$(x, s) \in \text{gph } \partial \mathcal{P}(f_1, f_2; \lambda_1, \lambda_2) \Leftrightarrow \begin{cases} x = \lambda_1 x_1 + \lambda_2 x_2, \\ s = x_1 + s_1 - x = x_2 + s_2 - x. \end{cases}$$

Proof. (i) From [14, Corollary XI.3.1.2], $\partial f_1(x) + \partial f_2(x) = \partial(f_1 + f_2)(x)$, and the result follows.

(ii) From [14, Corollary XI.3.4.2], $\partial f_1(x_1) \cap \partial f_2(x_2) = \partial(f_1 \square f_2)(x_1 + x_2)$, and the result follows.

(iii) From [5, Theorem 7.1], $\partial \mathcal{P}(f_1, f_2; \lambda_1, \lambda_2)(x) = -x + \cap_{i: \lambda_i > 0} (\partial f_i(x_i) + x_i)$, and the result follows. ■

Remark 12.3. An alternate proof of Lemma 12.1 can be obtained from Lemma 12.2(iii) using $f_1 = f$, $f_2 = q$, $\lambda_1 = 1 - \lambda$, and $\lambda_2 = \lambda$ to obtain $(x, s) \in \text{gph } T_\lambda$

if, and only if,

$$\begin{aligned} x &= (1 - \lambda)x_1 + \lambda x_2 \\ s &= x_1 + s_1 - x \\ s &= x_2 + s_2 - x. \end{aligned}$$

Then using $s_2 = x_2$ we can solve the system to obtain

$$\begin{aligned} x &= \left(1 - \frac{\lambda}{2}\right)x_1 + \frac{\lambda}{2}s_1 \\ s &= \frac{\lambda}{2}x_1 + \left(1 - \frac{\lambda}{2}\right)s_1, \end{aligned}$$

which is Formula (12.4).

12.4 Graph-Matrix Representation of PLQ Functions

The key idea is to use a data structure that allows the computation of the subdifferential using a single matrix multiplication. We store the subdifferential data of a PLQ function, and evaluate the function by integration.

The following data structure will be used. A lsc proper convex PLQ function f of one variable is defined by $f(x) = a_i x^2 + b_i x + c_i$ for $x \in [x_{i-1}, x_i]$ and $i \in \{1, \dots, n+1\}$, where $a_i, b_i \in \mathbb{R}$, $c_i \in \mathbb{R} \cup \{+\infty\}$, $x_i \in \mathbb{R} \cup \{-\infty, +\infty\}$ with $-\infty = x_0 < x_1 < \dots < x_n < x_{n+1} = +\infty$. While in previous work the function f was stored as the matrix

$$\begin{bmatrix} x_1 & a_1 & b_1 & c_1 \\ \vdots & \vdots & \vdots & \vdots \\ x_n & a_n & b_n & c_n \\ +\infty & a_{n+1} & b_{n+1} & c_{n+1} \end{bmatrix},$$

(which was called the PLQ matrix), we will use a different data structure.

To take advantage of the results in the previous section, the function f will be stored as a matrix

$$\begin{bmatrix} \bar{x}_0 & x_1 & \cdots & x_n & \bar{x}_{n+1} \\ \bar{s}_0 & s_1 & \cdots & s_n & \bar{s}_{n+1} \\ \bar{y}_0 & y_1 & \cdots & y_n & \bar{y}_{n+1} \end{bmatrix}.$$

The above matrix will be called a GPH matrix. We relax the requirement that $x_i < x_{i+1}$ in the PLQ matrix by only requiring $-\infty < \bar{x}_0 \leq x_1 \leq \dots \leq x_n \leq \bar{x}_{n+1} < +\infty$. The points x_i form a partition of the real line, the values s_i store the subdifferential data at the points x_i , and the values y_i store the value of f at x_i . Since the function f is piecewise quadratic, the graph of ∂f is piecewise affine. We store it as the piece-

wise affine graph going through (x_i, s_i) for $i = 1, \dots, n$ (with $s_i \in \partial f(x_i)$). (Since the graph may have vertical parts, it may not be a function). We capture the information outside $[x_1, x_n]$ by adding two points $(\bar{x}_0, \bar{s}_0)$ and $(\bar{x}_{n+1}, \bar{s}_{n+1})$ defined as follows.

Case 1: $\text{dom } f$ is unbounded from the left i.e., c_0 is finite. In that case, f is a quadratic function on $(-\infty, x_1]$. Then we define $\bar{x}_0 = x_1 - 1$, $\bar{s}_0 = 2a_0\bar{x}_0 + b_0$, and $\bar{y}_0 = f(\bar{x}_0) = a_0\bar{x}_0^2 + b_0\bar{x}_0 + c_0$.

Case 2: $\text{dom } f$ is bounded from the left i.e., $c_0 = +\infty$. In that case, $f(x) = +\infty$ for $x < x_1$. So we set $\bar{x}_0 = x_1$, $\bar{s}_0 = s_1 - 1$ and $\bar{y}_0 = +\infty$.

The definition of $(\bar{x}_{n+1}, \bar{s}_{n+1})$ is done similarly. These values correspond to a linear extension outside $[x_1, x_n]$ i.e., any subdifferential value on $(-\infty, x_1]$ (resp. $[x_n, +\infty)$) is on the half line going through $(\bar{x}_0, \bar{s}_0)$ and ending at (x_1, s_1) (resp. $(\bar{x}_{n+1}, \bar{s}_{n+1})$ and (x_n, s_n)). The point $\bar{x}_0$ (resp. $\bar{x}_{n+1}$) is added specifically to store the subdifferential on $(-\infty, x_1]$ (resp. $[x_n, +\infty)$).

Remark 12.4. The value 1 used in the definition of $(\bar{x}_0, \bar{s}_0)$ is arbitrary as any positive value can be used. To reduce floating point errors, a better value may be $x_2 - x_1$ in case 1 and $s_2 - s_1$ in case 2. The value 1 is used for simplicity.

To recover values of the function f , we can integrate the subdifferential but still need one constant of integration. For ease of computation, we store all the values $y_i = f(x_i)$ for $i = 1, \dots, n$. The additional values $\bar{y}_0$ and $\bar{y}_{n+1}$ are computed as above. Note that all values x_i and s_i are always finite for $i = 1, \dots, n$ as are $\bar{x}_0, \bar{s}_0, \bar{x}_{n+1}, \bar{s}_{n+1}$.

Example 12.5. The absolute value function may be stored as the matrix

$$\begin{bmatrix} -1 & 0 & 0 & 1 \\ -1 & -1 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix}.$$

The first and second row mean the graph of the subdifferential goes through the points $(-1, -1)$, $(0, -1)$, $(0, 1)$, and $(1, 1)$. The half-line going through $(-1, -1)$, and ending at $(0, -1)$ is extended to $(-\infty, -1]$, so the subdifferential is constant with value -1 on that interval. Similarly, it is constant with value 1 on $[1, +\infty)$. Finally, the first and last row give function values i.e., the function goes through the points $(-1, 1)$, $(0, 0)$, $(0, 0)$, and $(1, 1)$. Note that we store the point $(0, 0)$ twice for convenience.

Example 12.6. The function $f(x) = \max(0, |x| - 1)$ may be stored as the matrix

$$\begin{bmatrix} -2 & -1 & -1 & 1 & 1 & 2 \\ -1 & -1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Example 12.7. The function $f(x) = \max(0, x^2 - 1)$ may be stored as the matrix

$$\begin{bmatrix} -2 & -1 & -1 & 1 & 1 & 2 \\ -4 & -2 & 0 & 0 & 2 & 4 \\ 3 & 0 & 0 & 0 & 0 & 3 \end{bmatrix}.$$

There is no uniqueness of our representation. Values at x_i can be stored multiple times to capture multi-valuedness of the subdifferential and lower semicontinuity of the function, different points may be taken for $(\bar{x}_0, \bar{s}_0)$ and $(\bar{x}_{n+1}, \bar{s}_{n+1})$, and additional (redundant) points with abscissa not equal to x_i could be used in the GPH matrix. So different GPH matrices can represent the same function. While we could define a unique representation by imposing more constraints on the format, we would then have to normalize after each computation. By not normalizing we save computation time when calculating the composition of several operators. We also avoid coding a shape-preserving spline approximation algorithm. The downside is comparing two functions requires more work than a simple matrix comparison.

The following are “special” cases of our data structure. The function

$$\begin{bmatrix} -1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

is the function identically 0 (the values -1 and 1 can be replaced by any nonequal values). The indicator function $\iota_{[-1,1]}$ of the interval $[-1, 1]$ may be written

$$\begin{bmatrix} -1 & -1 & 1 & 1 \\ -1 & 0 & 0 & 1 \\ \infty & 0 & 0 & \infty \end{bmatrix}.$$

A quadratic function $f(x) = ax^2 + bx + c$ defined everywhere is stored as

$$\begin{bmatrix} \bar{x}_0 & \bar{x}_1 \\ 2a\bar{x}_0 + b & 2a\bar{x}_1 + b \\ a\bar{x}_0^2 + b\bar{x}_0 + c & a\bar{x}_1^2 + b\bar{x}_1 + c \end{bmatrix}$$

with $\bar{x}_0 < \bar{x}_1$. Conversely, the GPH matrix

$$\begin{bmatrix} \bar{x}_0 & \bar{x}_1 \\ \bar{s}_0 & \bar{s}_1 \\ \bar{y}_0 & \bar{y}_1 \end{bmatrix}$$

with $\bar{x}_0 < \bar{x}_1$ represents a quadratic function $f(x) = ax^2 + bx + c$ with $a = (\bar{s}_1 - \bar{s}_0)/(2(\bar{x}_1 - \bar{x}_0))$, $b = \bar{s}_0 - 2a\bar{x}_0$ and $c = \bar{y}_0 - a\bar{x}_0^2 - b\bar{x}_0 = \bar{y}_1 - a\bar{x}_1^2 - b\bar{x}_1$. Finally, the indicator function $\iota_{\{\bar{x}\}} + \bar{y}$ of a point has GPH matrix

$$\begin{bmatrix} \bar{x} & \bar{x} & \bar{x} & \bar{x} \\ \alpha - 1 & \alpha & \beta & \beta + 1 \\ \infty & \bar{y} & \bar{y} & \infty \end{bmatrix}$$

provided $\alpha < \beta$ (the value 1 can be replaced with any positive number).

12.5 Computational Convex Analysis through Graph Calculus

Using Formulas (12.3) with the GPH matrix gives simple algorithms for the most common convex operators. For example, the conjugate of the function represented as the GPH matrix $G = [x; s; y]$ where x , s , and y are row vectors, admits the GPH matrix

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \\ s.*x - y \end{bmatrix} * \begin{bmatrix} x \\ s \end{bmatrix} = \begin{bmatrix} s \\ x \\ s.*x - y \end{bmatrix},$$

where $.*$ represents the element-wise multiplication and $*$ the standard matrix multiplication. (Note that the operation $s.*x - y$ is always well defined as s_i and x_i are always finite even when $y_i = \infty$.)

Similarly, the scalar multiplication by $\alpha > 0$ of the same function represented by G has GPH matrix

$$\begin{bmatrix} 1 & 0 \\ 0 & \alpha \\ \alpha y \end{bmatrix} * \begin{bmatrix} x \\ s \end{bmatrix} = \begin{bmatrix} x \\ \alpha s \\ \alpha y \end{bmatrix}.$$

The Moreau envelope can be computed as follows.

Proposition 12.8. *Given a convex PLQ function f in GPH matrix G given by Formula (12.1), its Moreau envelope $e_\lambda f$ admits the GPH matrix*

$$\begin{bmatrix} 1 & \lambda \\ 0 & 1 \\ f + \frac{\lambda}{2}s.\hat{2} \end{bmatrix} * \begin{bmatrix} x \\ s \end{bmatrix} = \begin{bmatrix} x + \lambda s \\ s \\ f + \frac{\lambda}{2}s.\hat{2} \end{bmatrix},$$

where $\hat{.}$ is the elementwise square function.

Proof. Using the fact $e_\lambda f = (f^* + \lambda q)^*$ and Formulas (12.3) we deduce that f^* admits the GPH matrix $[s; x; s.*x - f]$ so the function $f^* + \lambda q$ can be represented as $[s; x + \lambda s; s.*x - f + \frac{\lambda}{2}s.\hat{2}]$. Taking the conjugate gives the GPH matrix of the Moreau envelope and finishes the proof. ■

We next compute a GPH matrix of the proximal average.

Proposition 12.9. *Given two convex PLQ functions f_1 (resp. f_2) in GPH matrix $G_1 = [x_1; s_1; y_1]$ (resp. $G_2 = [x_2; s_2; y_2]$) and $\lambda_1 = 1 - \lambda$, $\lambda_2 = \lambda$ with $\lambda \in [0, 1]$;*

the λ proximal average of f_1 with f_2 admits the GPH matrix $G = [x; s; y]$, where $x = \lambda_1 x_1 + \lambda_2 P x_1$, $s = x_1 + s_1 - x$,

$$y = \lambda_1 \left(y_1 + \frac{1}{2} x_1 \cdot \cdot 2 \right) + \lambda_2 \left(y_{P x_1} + \frac{1}{2} P x_1 \cdot \cdot 2 \right) - \frac{1}{2} x \cdot \cdot 2$$

the operator P is defined by

$$P = (\text{Id} + \partial f_2)^{-1} (\text{Id} + \partial f_1),$$

and $y_{P x_1} = f_2(P x_1)$.

Proof. We parametrize the graph of the proximal average using x_1 , so we need to find the corresponding x_2 for each value of x_1 . From Lemma 12.2 we have $x_1 - x + s_1 = x_2 - x + s_2$ so we deduce $(\text{Id} + \partial f_1)(x_1) = (\text{Id} + \partial f_2)(x_2)$, thus $x_2 = P x_1$ is the corresponding point that makes the prox average exact. The value of y comes from Formula (12.2). ■

We note that Proposition 12.9 allows us to speed up computation by precomputing $P x_1$, $(y_1 + \frac{1}{2} x_1 \cdot \cdot 2)$ and $(y_{P x_1} + \frac{1}{2} P x_1 \cdot \cdot 2)$, which do not depend on λ . Then we can deduce the λ -dependent values x , s , and y . Applying such precomputation scheme, we plot the proximal average of $f_0(x) = x^2/2$ with $f_1(x) = \iota_{\{0\}}(x) + 1$ (this example was challenging to plot during the writing of [4] and motivated further research to speed up such computation [16, 22]) in Fig. 12.1. The PLQ algorithm took 117s to generate the picture under Scilab 5.1.1, while with precomputation, using the GPH algorithm only took 33 s.

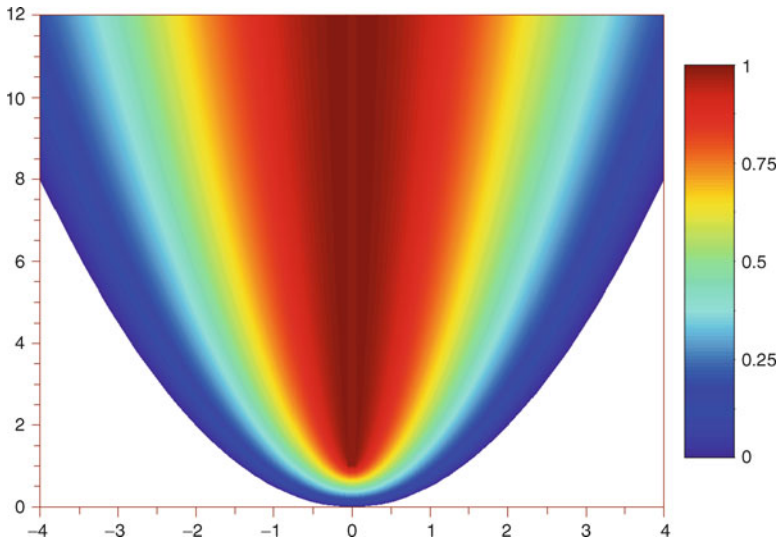


Fig. 12.1 The proximal average of $f_0(x) = x^2/2$ with $f_1(x) = \iota_{\{0\}}(x) + 1$. Each color corresponds to a specific value of $\lambda \in [0, 1]$

12.6 Comparing GPH Algorithms with PLQ Algorithms

The previous section contains extremely simple algorithms to compute the conjugate, the scalar multiplication, the Moreau envelope, and the proximal average. In particular, the computation of the conjugate is much simpler than previous algorithms, namely the (obsolete log-linear time) fast Fenchel transform [7, 9, 17, 25], and the (optimal) linear time algorithms: the LLT [18], the PLQ algorithm [22], the parametric Legendre transform (PLT) [15], and the parabolic envelope algorithm (PE) [10].

All numerical experiments were performed on an IBM Thinkpad Lenovo T60 running Intel Core Duo at 2Ghz with 2GB memory using Scilab 5.1.1 under Windows XP pro SP3.

In [19], we argued for the superiority of the PLQ algorithm over the LLT algorithm based on the ability of PLQ algorithms to model unbounded effective domains, and the closedness of the class of PLQ functions under common convex operators which makes the algorithms exact for these functions. The price to pay for these advantages is the more expensive computation time required by the PLQ algorithms over the LLT algorithm. The GPH algorithms bridge that gap by keeping all the properties of the PLQ algorithm with the speed of the LLT algorithm. Figure 12.2a shows the CPU time vs. n (the number of pieces) when computing the conjugate with the PLQ, GPH, and LLT algorithms: the PLQ algorithm is slower, while the GPH and LLT algorithms run in almost the same time. We included in Fig. 12.2b the computation time using a nonsmooth bundle method (n1fc1 as available through the `optim` function in Scilab) at each point warmstarted by the value at the previous point. The figure clearly shows that specialized algorithms PLQ, GPH, and LLT outperform a generic optimization solver.

The numerical computation of the Moreau envelope is compared in Fig. 12.3. The algorithms LLT and GPH share similar performances and are faster than the PLQ algorithm, which is itself faster than the OPT algorithm.

The numerical computation of the proximal average envelope is compared in Fig. 12.4. The graph clearly shows the GPH algorithm is faster (without precomputation as it does not apply here) and that for $n = 700$ the GPH algorithm runs 15 times faster than the PLQ algorithm and 95 times faster than the OPT algorithm.

We conclude this section with a couple of examples illustrating the capabilities of the numerical package. Figure 12.5 shows the subdifferential of a piecewise affine function. Adding the GPH algorithms expanded the existing numerical library to manipulate subdifferentials. Finally Fig. 12.6 illustrates the proximal point algorithm. It shows the proximal mapping associated with the quadratic function $f(x) = x^2$ computed using graph-matrix calculus, the line $y = x$, and the iterations of the proximal point algorithm. We clearly see convergence to the minimum of the function which is the only fixed point.

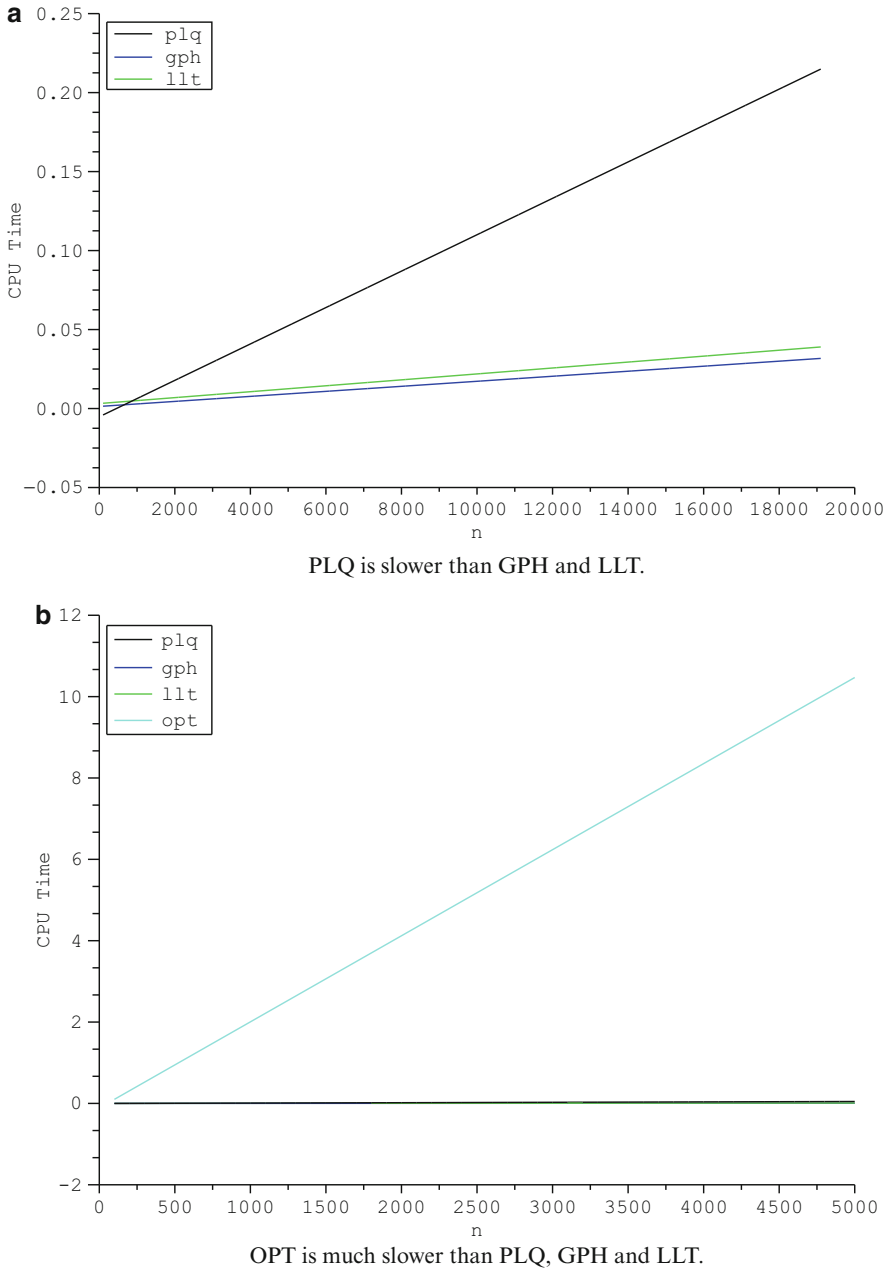


Fig. 12.2 Comparison of PLQ, GPH, and LLT algorithms for computing the conjugate of the function $f(x) = x^4$ approximated by n piecewise linear functions on $[-10, 10]$ (Least-square regression lines are shown). (a) GPH and LLT are on par while PLQ is slower. (b) OPT is much slower than the other three algorithms

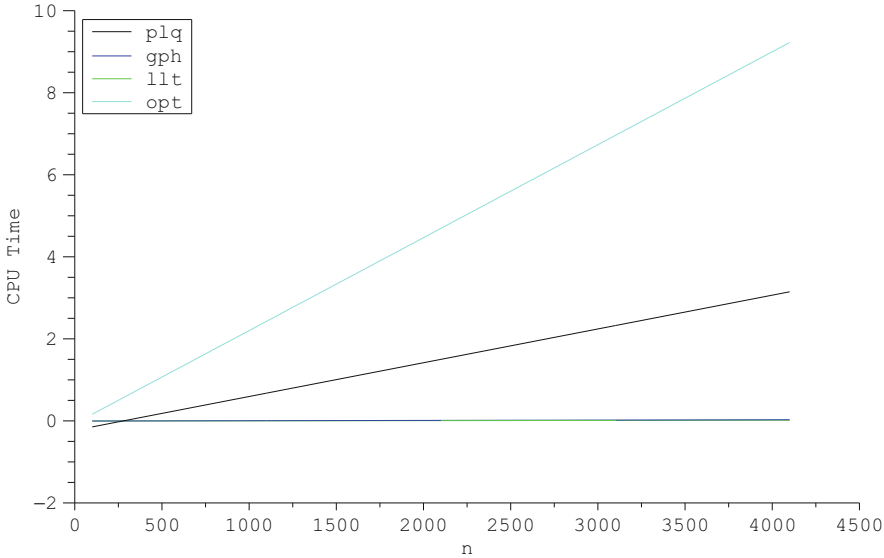


Fig. 12.3 Comparison of PLQ, GPH, LLT, and OPT algorithms for computing the Moreau envelope of the function $f(x) = x^4$ approximated by n piecewise linear functions on $[-10, 10]$ when $\lambda = 0.5$ (Least-square regression lines are shown)

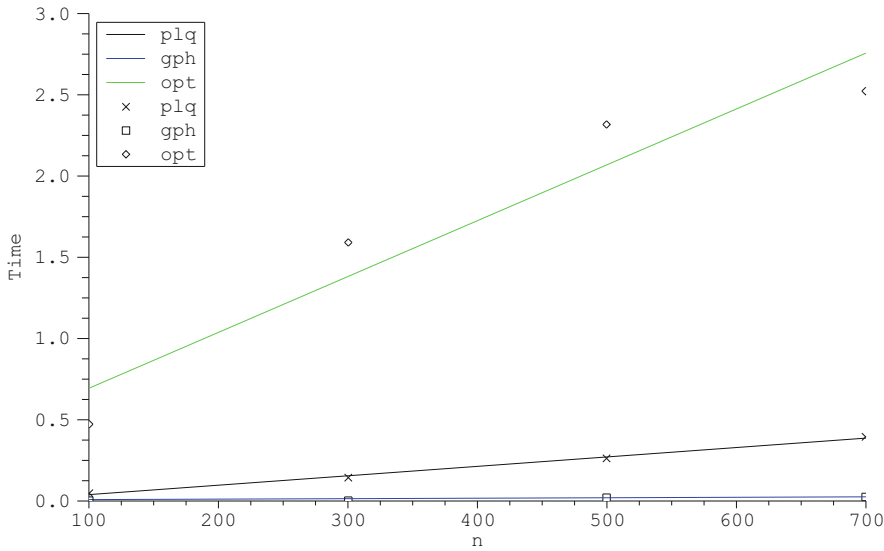


Fig. 12.4 Comparison of PLQ, GPH, and OPT algorithms for computing the proximal average of the function $f_1(x) = x^4$ with $f_2(x) = e^x$ approximated by n piecewise affine functions on $[-10, 10]$ when $\lambda = 0.5$ (Least-square regression lines are shown)

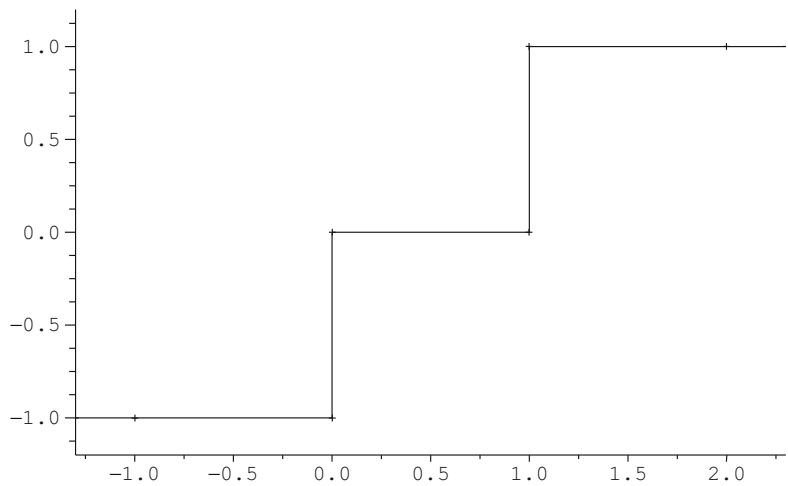


Fig. 12.5 Subdifferential of the function $f(x) = -x$ for $x \leq 0$, $f(x) = 0$ when $0 \leq x \leq 1$, and $f(x) = x - 1$ if $x \geq 1$. The points indicated belong to the GPH matrix of f

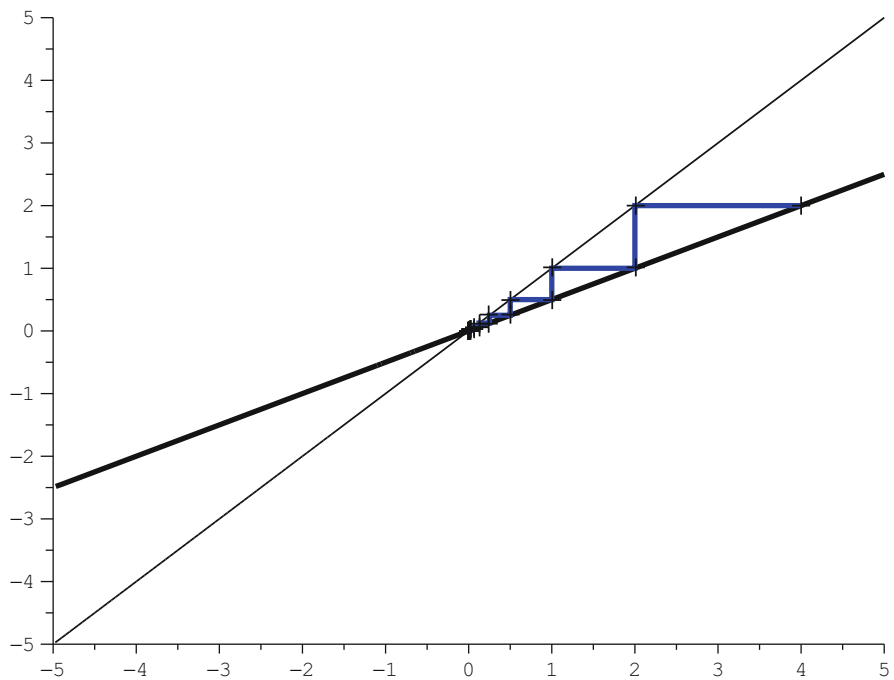


Fig. 12.6 Proximal mapping of the function $f(x) = x^2$ with the 16 iterations of the proximal point algorithm

12.7 Conclusion

We recalled Goebel's graph-matrix calculus formulas that relate the graph of the subdifferential of convex operators to the graph of the subdifferential of the function it is applied to. We proposed an intuitive data structure for storing PLQ functions in GPH format and deduced very simple linear-time algorithms to compute numerically the main convex operators. In addition, we provided numerical evidence that the algorithms are (an order of magnitude) faster than previously known PLQ algorithms while providing the same modeling advantages. We also showed that the PLQ, LLT, and GPH specialized algorithms are much faster than using a generic solver with warmstarting (named the OPT algorithms). The new GPH algorithms extend the current CCA numerical library by providing a natural manipulation of the subdifferential. They are especially efficient in matrix-based languages like Matlab and Scilab.

Future work will focus on extending the algorithms to handle nonconvex functions, and to provide a numerical library to manipulate bivariate functions.

Acknowledgements The authors thank the two referees for their careful reading of the manuscripts and their multiple comments, which resulted in correcting an error in Lemma 12.4.

Yves Lucet was partially supported by a Discovery grant from the Natural Sciences and Engineering Research Council of Canada.

References

1. Bauschke, H.H., Wang, X.: The kernel average of two convex functions and its application to the extension and representation of monotone operators. *Trans. Amer. Math. Soc.* **361**, 5947–5965 (2009)
2. Bauschke, H.H., Matoušková, E., Reich, S.: Projection and proximal point methods: Convergence results and counterexamples. *Nonlinear Anal.* **56**, 715–738 (2004)
3. Bauschke, H.H., Lucet, Y., Wang, X.: Primal-dual symmetric intrinsic methods for finding antiderivatives of cyclically monotone operators. *SIAM J. Control Optim.* **46**, 2031–2051 (2007)
4. Bauschke, H.H., Lucet, Y., Trienis, M.: How to transform one convex function continuously into another. *SIAM Rev.* **50**, 115–132 (2008)
5. Bauschke, H.H., Goebel, R., Lucet, Y., Wang, X.: The proximal average: Basic theory. *SIAM J. Optim.* **19**, 768–785 (2008)
6. Bauschke, H.H., Moffat, S.M., Wang, X.: Self-dual smooth approximations of convex functions via the proximal average. Tech. Rep. arXiv:1003.5866v1 [math.FA], UBC Okanagan (2010)
7. Brenier, Y.: Un algorithme rapide pour le calcul de transformées de Legendre–Fenchel discrètes. *C. R. Acad. Sci. Paris Sér. I Math.* **308**, 587–589 (1989)
8. Computational Convex Analysis library. <https://people.ok.ubc.ca/ylucet/ccs.html> (1996–2009)
9. Corrias, L.: Fast Legendre–Fenchel transform and applications to Hamilton–Jacobi equations and conservation laws. *SIAM J. Numer. Anal.* **33**, 1534–1558 (1996)
10. Felzenszwalb, P.F., Huttenlocher, D.P.: Distance transforms of sampled functions. Tech. Rep. TR2004-1963, Cornell Computing and Information Science (2004)
11. Gardiner, B., Lucet, Y.: Numerical computation of Fitzpatrick functions. *J. Convex Anal.* **16**, 779–790 (2009)
12. Goebel, R.: Self-dual smoothing of convex and saddle functions. *J. Convex Anal.* **15**, 179–190 (2008)

13. Hare, W.: A proximal average for nonconvex functions: A proximal stability perspective. *SIAM J. Optim.* **20**, 650–666 (2009)
14. Hiriart-Urruty, J.B., Lemaréchal, C.: *Convex Analysis and Minimization Algorithms, Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences]*, vol. 305–306. Springer, Berlin (1993)
15. Hiriart-Urruty, J.B., Lucet, Y.: Parametric computation of the Legendre–Fenchel conjugate. *J. Convex Anal.* **14**, 657–666 (2007)
16. Koch, V., Johnstone, J., Lucet, Y.: Convexity of the proximal average. Tech. rep., University of British Columbia (2010). Accepted for publication in *Journal of Optimization Theory and Applications*
17. Lucet, Y.: A fast computational algorithm for the Legendre–Fenchel transform. *Comput. Optim. Appl.* **6**, 27–57 (1996)
18. Lucet, Y.: Faster than the Fast Legendre Transform, the Linear-time Legendre Transform. *Numer. Algorithms* **16**, 171–185 (1997)
19. Lucet, Y.: Fast Moreau envelope computation I: Numerical algorithms. *Numer. Algorithms* **43**, 235–249 (2006)
20. Lucet, Y.: New sequential exact Euclidean distance transform algorithms based on convex analysis. *Image and Vision Computing* **27**, 37–44 (2009)
21. Lucet, Y.: What shape is your conjugate? A survey of computational convex analysis and its applications. *SIAM J. Optim.* **20**, 216–250 (2009)
22. Lucet, Y., Bauschke, H.H., Trienis, M.: The piecewise linear-quadratic model for computational convex analysis. *Comput. Optim. Appl.* **43**, 95–118 (2009)
23. Moffat, S.M.: On the kernel average of n functions. Master’s thesis, Department of Mathematics, University of British Columbia (2009)
24. Moreau, J.J.: Proximité et dualité dans un espace Hilbertien. *Bull. Soc. Math. France* **93**, 273–299 (1965)
25. Noullez, A., Vergassola, M.: A fast Legendre transform algorithm and applications to the adhesion model. *J. Sci. Comput.* **9**, 259–281 (1994)
26. Rockafellar, R.T., Wets, R.J.B.: *Variational Analysis*. Springer, Berlin (1998)
27. She, Z.S., Aurell, E., Frisch, U.: The inviscid Burgers equation with initial data of Brownian type. *Comm. Math. Phys.* **148**, 623–641 (1992)